UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

BACHELOR THESIS

By
Phung Dam Tien Si - 23BI14384

Title:
Design and Implementation of a Full-Stack Web-Based Recruitment Platform for Students and Employers

External Supervisor: Eng. Dinh Viet Ha

Internal Supervisor: Prof. Dr. Giang Anh Tuan

Hanoi, June - 2026

To whom it may concern,

I, ... (supervisor's name), certify that the thesis/ internship report of Mr/Ms. ..... (student's name) is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, <date>

Supervisor's signature

Acknowledgements

I would like to express my sincere gratitude to Mr. Dinh Viet Ha for his guidance, patience and mentorship throughout my internship. His support have greatly contributed to the completion of this project and my professional development.

I would also like to extend my sincere appreciation to SAVA META Technology Joint Stock Company for providing me with the opportunity to undertake my internship in a professional and supportive working environment.

Finally, I would like to thank everyone who has supported, encouraged, and accompanied me throughout my academic journey and personal development. Their encouragement has been a constant source of motivation during the completion of this thesis.

Contents

List of Figures

List of Tables

Abbreviations

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, and Delete
CV	Curriculum Vitae
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
WDLC	Web Development Life Cycle

Chapter 1

Introduction

Chapter 2

Literature Review

Chapter 3

System Analysis

3.1 Problem Statement

The rapid growth of Internet technologies has significantly transformed the recruitment process. Traditional methods, such as submitting printed resumes directly to companies, have gradually been replaced by online recruitment platforms that enable employers to publish vacancies and job seekers to submit applications electronically. This digital transformation has reduced geographical barriers and accelerated communication between employers and candidates.

However, the increasing number of online recruitment services has also introduced new challenges for job seekers. While online platforms provide greater access to employment opportunities, users often need to navigate different interface designs, search mechanisms, and application workflows across multiple websites. This fragmentation may increase the learning curve for new users and require them to maintain personal profiles and resumes records across multiple recruitment platforms, making the job search process less efficient.

Employers face similar challenges when managing recruitment activities across numerous candidates. Recruitment involves publishing job vacancies, reviewing applications, communicating hiring decisions, and maintaining up-to-date recruitment records. Performing these tasks efficiently becomes increasingly difficult without a centralized platform that organizes recruitment information and supports interactions between employers and job seekers.

These challenges motivate the development of a centralized recruitment platform. A web-based recruitment platform provides a centralized environment where job seekers can search for employment opportunities, maintain professional profiles, upload resumes, and track submitted applications. At the same time, employers can manage company information, publish job postings, and review applications through a unified interface.

Since the proposed system stores sensitive personal information and supports operations that modify data, authentication is required to verify user identity before granting access to protected resources. After authentication, users can securely update their personal profiles, resumes, job applications, or company information according to their permissions.

Furthermore, different categories of users require different system capabilities. The system adopts role-based access control to ensure that each user can only access functions relevant to their responsibilities. Job Seekers are responsible for maintaining personal

profiles and applying for jobs. Employers manage company profiles, publish job postings, and review applications. Administrators supervise the platform by managing user accounts and moderating job postings. Separating permissions according to user roles improves both system security and usability.

3.2 Functional Requirement

Functional requirements define the core functionalities that the proposed recruitment platform must provide in order to satisfy the needs of its stakeholders. These requirements describe the services and operations that the system is expected to perform for different categories of users. For clarity and readability, the functional requirements are organized into major functional modules based on their business responsibilities, as summarized in Table 3.1.

Table 3.1: Summary of Functional Requirements.

Module	**Description**
Authentication	Supports user registration, login, logout, and password recovery to ensure secure access to the system.
User Profile Management	Allows job seekers to maintain personal information, update job preferences, and manage uploaded resumes.
Job Discovery	Provides browsing, searching, filtering, and viewing of available job postings.
Job Application	Enables job seekers to submit applications, withdraw applications before processing, and monitor application status.
Company Management	Allows employers to create and maintain company information presented to prospective applicants.
Job Posting Management	Allows employers to create, edit, publish, close, and manage job postings throughout their recruitment lifecycle.
Recruitment Management	Supports employers in reviewing submitted applications and updating the recruitment status of candidates.
Administration	Allows administrators to manage user accounts, moderate job postings, and maintain overall platform integrity.

The proposed recruitment platform is organized into eight major functional modules that collectively support the complete recruitment workflow. These modules cover the essential operations required by job seekers, employers, and administrators, while providing a clear separation of responsibilities within the system.

3.3 Non-Functional Requirement

Non-functional requirements define the quality attributes and operational constraints that influence how the proposed recruitment platform performs its functions. These

requirements focus on aspects such as security, performance, usability, reliability, maintainability, and compatibility. The major non-functional requirements considered during the system design are summarized in Table 3.2.

Table 3.2: Summary of Non-Functional Requirements.

Category	**Description**
Security	The system shall authenticate users before granting access to protected resources and enforce role-based authorization. Sensitive credentials shall be securely stored using password hashing techniques.
Performance	The system shall provide responsive access to common operations such as browsing job postings, submitting applications, and managing recruitment information.
Usability	The system shall provide a responsive web interface with consistent navigation, clear validation messages, and an intuitive user experience for all supported roles.
Reliability	The system shall ensure data consistency through proper validation, controlled data modification, and appropriate error handling mechanisms.
Maintainability	The system shall adopt a modular architecture with clearly separated components to simplify future maintenance and feature extension.
Compatibility	The system shall support modern desktop web browsers without requiring additional client-side software installation.

The non-functional requirements establish the quality objectives of the proposed system and serve as guiding principles throughout the system design and implementation. Satisfying these requirements helps ensure that the platform is secure, reliable, maintainable, and capable of providing a consistent user experience.

3.4 Use Case Analysis

Use case analysis identifies the interactions between system users and the proposed recruitment platform. By modeling these interactions, the functional scope of the system can be clearly illustrated from the perspective of different user roles. This section first presents the shared use cases that are common across multiple actors before discussing the specialized use cases associated with each primary user role.

The proposed recruitment platform supports three primary user roles: Job Seeker, Employer, and Administrator. Each role performs different business functions and therefore interacts with a different subset of the system.

3.4.1 Shared Use Cases

To simplify the Use Case Model, the actors are organized using UML actor generalization. Users who have successfully signed in are represented by the generalized actor Authenticated User, while unauthenticated visitors are represented by the Guest actor.

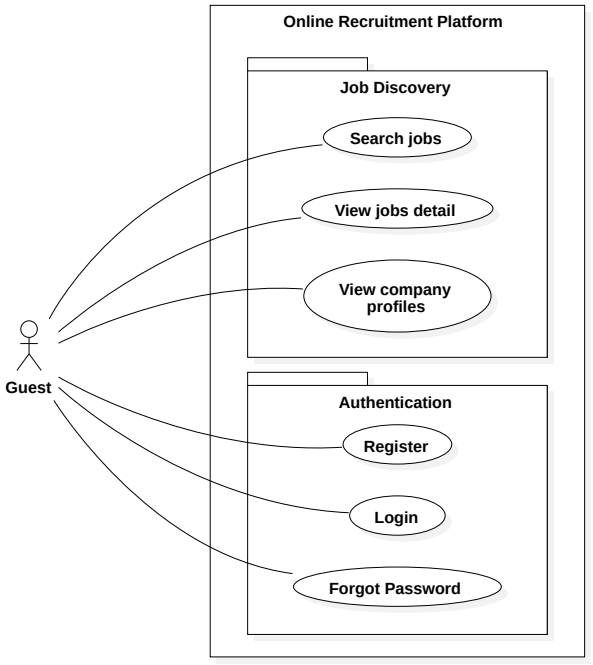

Figure 3.1: Guest Use Case Diagram.

As illustrated in Figure 3.1, Guest users are limited to publicly accessible functions such as browsing available job postings, viewing company information, and creating a new account. These use cases provide basic system accessibility without requiring authentication while encouraging users to register before accessing personalized recruitment features.

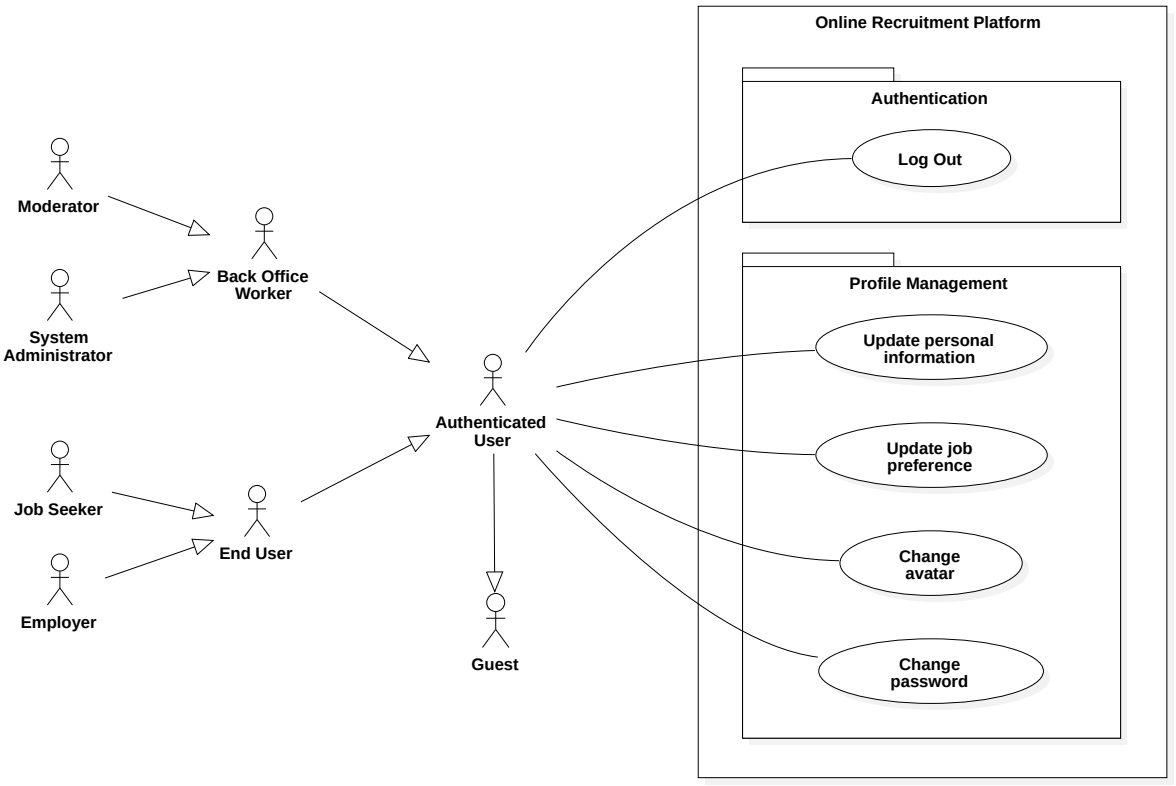

Figure 3.2: Authenticated User Use Case Diagram.

Figure 3.2 illustrates the common functionalities available to all authenticated users regardless of their assigned role. These shared use cases include authentication-related operations such as updating account information, changing passwords, and logging out. Additional business-specific functions are introduced through the specialized actors discussed in the following sections.

3.4.2 Specialized Use Cases

The following subsections present the business-specific use cases associated with each primary user role.

Job Seeker Use Cases

The Job Seeker represents the primary user of the recruitment platform. This actor is responsible for searching employment opportunities, maintaining personal information, and participating in the recruitment process through job applications.

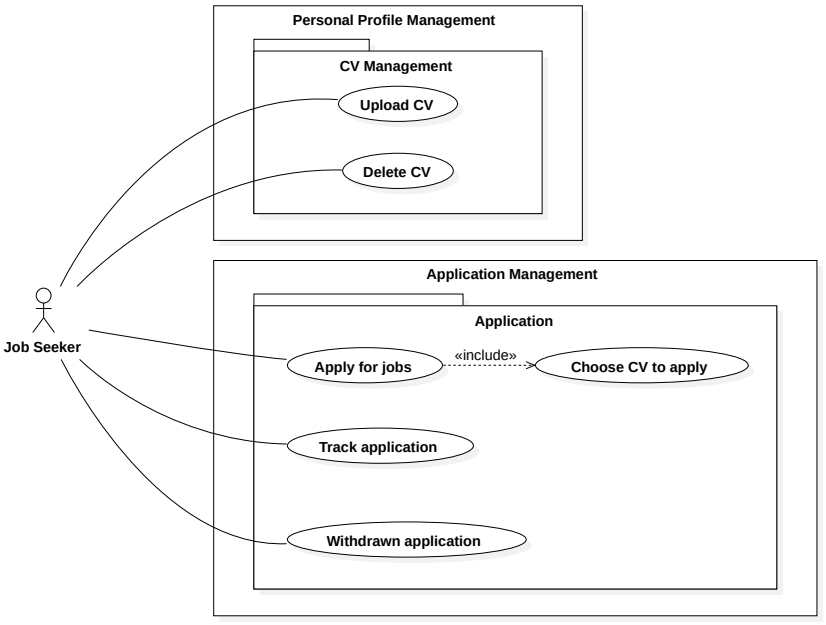

Figure 3.3: Job Seeker Use Case Diagram.

As shown in Figure 3.3, the Job Seeker interacts with the system through two major functional areas: profile and job application management.

Employer Use Cases

The Employer actor represents organizations or recruiters responsible for publishing job postings and evaluating applicants.

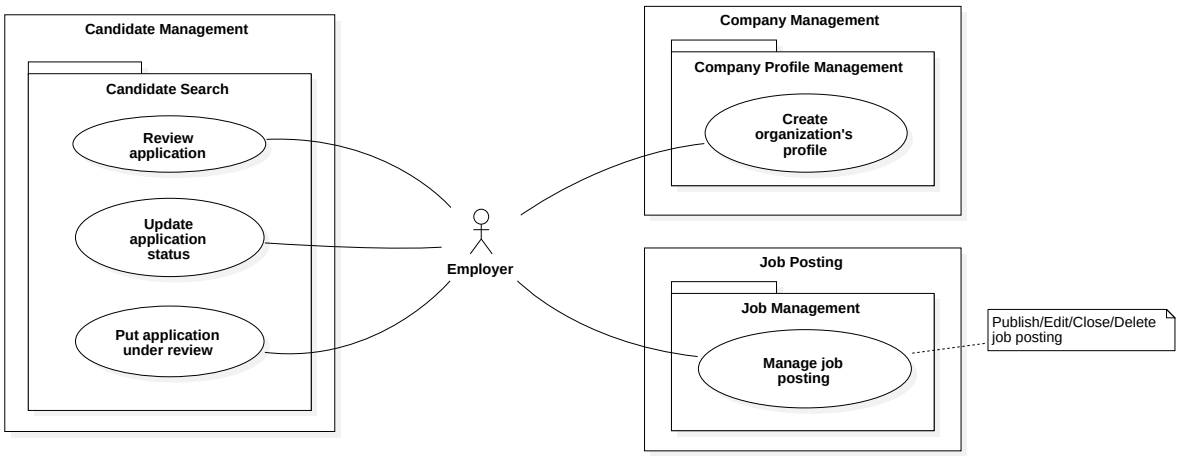

Figure 3.4: Employer Use Case Diagram.

Illustrated in Figure 3.4, the Employer's use cases focus on company management, job posting management, and recruitment management. Together, these functions support the complete recruitment lifecycle, from creating a company profile to reviewing applications and updating recruitment status.

Back Office Worker Use Cases

The Back Office Worker represents internal personnel responsible for maintaining platform integrity and ensuring that the recruitment platform operates correctly.

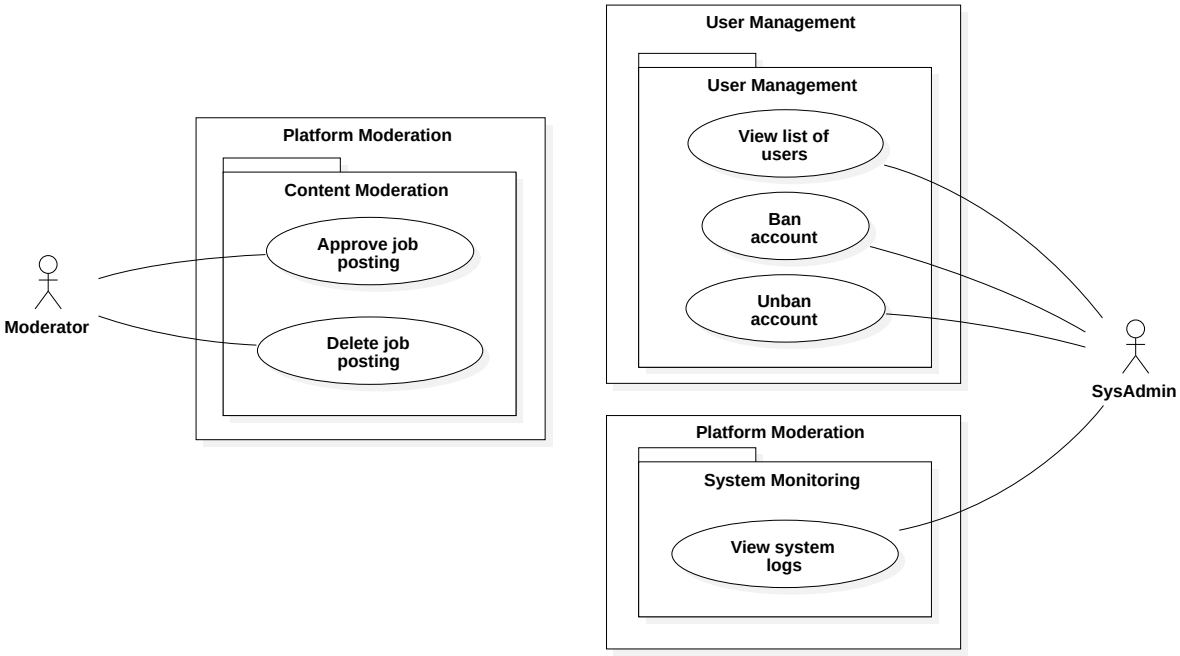

Figure 3.5: Back Office Worker Use Case Diagram.

Unlike Job Seekers and Employers, the Back Office Workers primarily performs supervisory tasks highlighted in Figure 3.5. These include user management, moderation of job postings, and maintaining the overall quality and security of the platform.

3.5 Use Case Specification

The Use Case Diagrams presented in the previous section provide an overview of the functional scope of the proposed recruitment platform. This section further illustrates the system behavior by presenting detailed specifications for selected use cases that represent the primary functionalities performed by different user roles.

3.5.1 Guest: Login

Table 3.3: Use Case Specification: Login.

Field	**Specification**
Use Case ID	UC-AUTH-02
Use Case Name	Login
Primary Actor	Guest
Description	Allows a guest to authenticate using registered credentials and become an authenticated user.
Goal	Authenticate a registered user and grant access to protected system functions.
Preconditions	• The user has a registered account. • The user is not currently authenticated.
Trigger	The user clicks "Log In" button from the website's header
Main Success Scenario	1. Guest selects "Login" 2. System displays the login form. 3. Guest enters email and password. 4. Guest submits the login form. 5. System validates the submitted credentials. 6. System authenticates the user. 7. System establishes an authenticated session. 8. System redirects the user to the authenticated homepage. 9. System displays a successful login message.

Table 3.3 (Continued)

Field	Specification
Alternative Flows	**A1. Invalid credentialsd** 1. The system reports the error "Invalid email or password". **A2. Invalid email address** 1. The system reports the error "Invalid email". **A3. Missing required input fields** 1. The system reports the error "Please fill out required fields". **A4. Account is suspended** 1. System denies authentication. 2. System informs the user that the account has been suspended.
Postconditions	• The user is successfully authenticated. • The user gains access to functions available to authenticated users.

3.5.2 Authenticated User: Update Personal Information

Table 3.4: Use Case Specification: Update Personal Information.

Field	**Specification**
Use Case ID	UC-PROFILE-01
Use Case Name	Update Personal Information
Primary Actor	Authenticated User
Goal	Maintain accurate personal information within the platform.
Description	Allows an authenticated user to update personal profile information.
Preconditions	• User is authenticated.
Trigger	The user selects the Profile page.
Main Success Scenario	1. User opens the profile page. 2. System displays current profile information. 3. User edits profile information. 4. User submits the changes. 5. System validates the submitted information. 6. System updates the profile. 7. System confirms the successful update.
Alternative Flows	**A1. Invalid input** 1. System highlights invalid fields. 2. User corrects the information.
Postconditions	• Updated profile information is stored.

3.5.3 Job Seeker: Apply for Job

Table 3.5: Use Case Specification: Apply for Job.

Field	**Specification**
Use Case ID	UC-JS-01
Use Case Name	Apply for Job
Primary Actor	Job Seeker
Goal	Submit a job application to an employer.
Description	Allows a Job Seeker to submit an application for a published job posting.
Preconditions	• Job Seeker is authenticated. • Job posting is open for applications. • Job Seeker has an uploaded resume.
Trigger	The Job Seeker selects the **Apply** option on a job posting.
Main Success Scenario	1. Job Seeker views a job posting. 2. Job Seeker selects Apply. 3. System displays the application form. 4. Job Seeker selects a resume. 5. Job Seeker submits the application. 6. System validates the submission. 7. System records the application. 8. System confirms successful submission.
Alternative Flows	**A1. Resume unavailable** 1. System requests the Job Seeker to upload a resume. **A2. Already applied** 1. System rejects the submission. 2. System informs the Job Seeker that an application already exists.
Postconditions	• A new job application is created.

3.5.4 Employer: Publish Job Posting

Table 3.6: Use Case Specification: Publish Job Posting.

Field	**Specification**
Use Case ID	UC-EMP-01
Use Case Name	Publish Job Posting
Primary Actor	Employer
Goal	Publish a job posting for potential applicants.
Description	Allows an Employer to create and publish a new job posting.
Preconditions	• Employer is authenticated. • Employer has a registered company profile.
Trigger	Employer selects the Create Job Posting function.
Main Success Scenario	1. Employer enters job information. 2. Employer submits the job posting. 3. System validates the submitted information. 4. System creates the job posting. 5. System publishes the job posting. 6. System confirms successful publication.
Alternative Flows	**A1. Invalid job information** 1. System identifies invalid fields. 2. Employer updates the information.
Postconditions	• Job posting becomes visible to Job Seekers.

3.5.5 Back Office Worker: Approve Job Posting

Table 3.7: Use Case Specification: Approve Job Posting.

Field	**Specification**
Use Case ID	UC-ADMIN-01
Use Case Name	Approve Job Posting
Primary Actor	Back Office Worker
Goal	Review and approve job postings before they become publicly available.
Description	Allows an Back Office Worker to review submitted job postings and determine whether they comply with platform policies.
Preconditions	• Back Office Worker is authenticated. • A job posting is awaiting approval.
Trigger	Administrator opens the pending job posting list.
Main Success Scenario	1. Back Office Worker reviews the job posting. 2. Back Office Worker verifies the submitted information. 3. Back Office Worker approves the posting. 4. System updates the posting status. 5. System makes the job posting publicly available.
Alternative Flows	**A1. Job posting rejected** 1. Administrator rejects the posting. 2. System records the decision. 3. System notifies the Employer.
Postconditions	• The job posting is approved or rejected.

Chapter 4

System Design

4.1 Design Overview

This chapter presents the technical design of the proposed system based on the requirements identified in Chapter 3. It describes the system architecture, database design, backend structure, and frontend organization that form the blueprint for the implementation discussed in Chapter 5.

The design follows a small set of guiding principles, each addressing one or more non-functional requirements identified previously. Table 4.1 summarizes these principles and their rationale.

Table 4.1: Design Principles and Rationale.

Principle	**Rationale**
Layered Architecture	Separates presentation, API, business logic, and data access concerns, improving maintainability and testability.
Feature-Oriented Modular Design	Organizes code by business domain (e.g., Profile, Job, Application) rather than by technical type, keeping related logic together and easing future extension.
Thin Controller, Rich Service	Controllers handle only request/response concerns; business logic and validation reside in services, reducing duplication and improving reusability.
Security by Design	Authentication and authorization are consistently enforced to protect restricted resources, while credentials are securely stored using industry-standard cryptographic techniques.
Data Integrity	Relational constraints, foreign keys, and database integrity rules ensure data consistency and prevent invalid relationships.

These principles are applied consistently across all modules of the system, including those where implementation is still in progress at the time of writing (e.g., Employer and Back Office Worker features), ensuring maintainability and reduces coupling between components for future development.

The remainder of this chapter is organized as follows. Section 4.2 presents the system architecture, including the C4-based container view. Section 4.3 details the database design and entity-relationship model. Section 4.4 describes the backend design, including the layered structure and module organization illustrated through the Profile module. Section 4.5 covers the frontend design. Section 4.6 summarizes the chapter.

4.2 System Architecture

The system architecture is documented using the C4 model, which describes a software system at progressively narrower levels of zoom: Context, Container, and Component. This progression matches the layered, modular design principles established in Section 4.1, letting each stakeholder read only the level of detail relevant to them.

4.2.1 System Context

The System Context diagram identifies the system's boundary and its interactions with external actors, without describing internal structure.

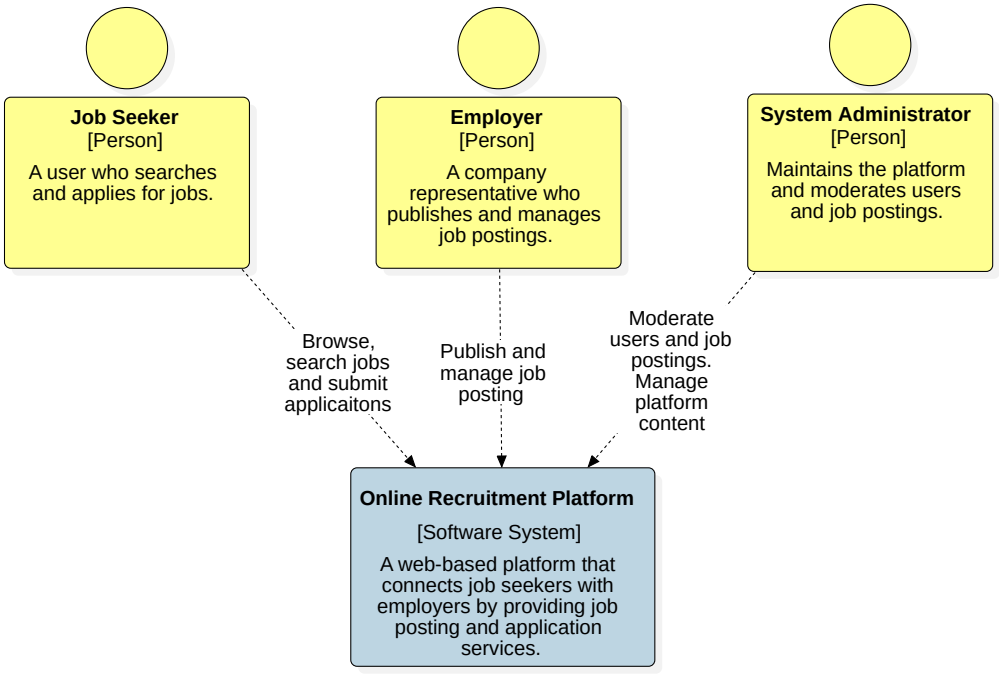

Figure 4.1: System Context Diagram.

As shown in Figure 4.1, three actors interact with the Online Recruitment Platform: the Job Seeker browses and applies for jobs, the Employer publishes and manages job postings, and the System Administrator moderates users and platform content. These actors correspond directly to the roles established in the use case analysis in Chapter 3.

4.2.2 Container View

The Container diagram decomposes the system into its major deployable units and shows how they communicate.

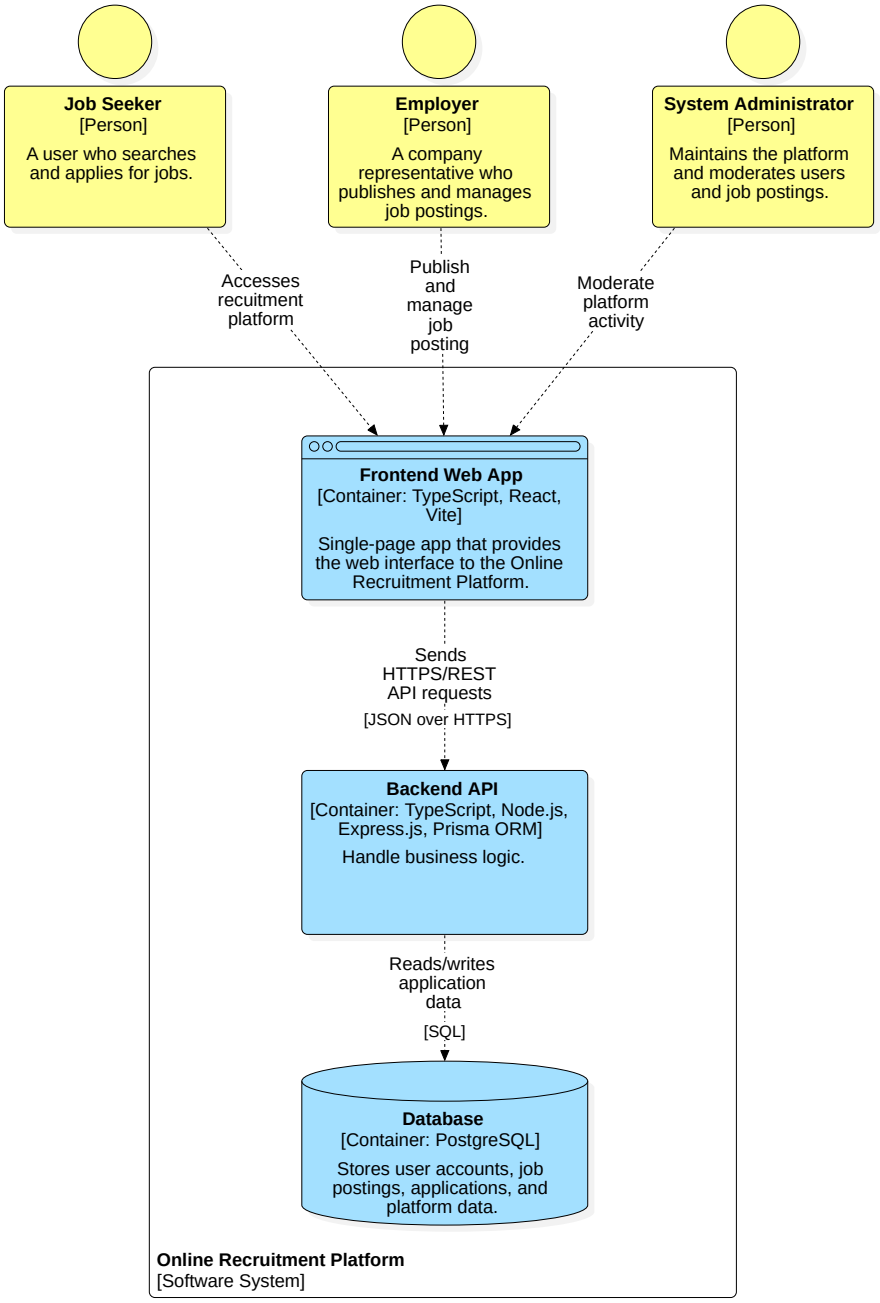

Figure 4.2: Container Diagram.

As illustrated in Figure 4.2, the system consists of three containers:

- The Frontend Web App is a React single-page application that renders the user interface.
- The Backend API is a Node.js/Express service, built with TypeScript and Prisma ORM, that implements all business logic and exposes it as a REST API.

- The Database is a PostgreSQL instance that persists user accounts, job postings, applications, and other platform data.

The Frontend communicates with the Backend over HTTPS using JSON, and the Backend communicates with the Database using SQL through Prisma.

4.2.3 Component View: Frontend

The Frontend component diagram decomposes the Frontend Web App container into its internal building blocks.

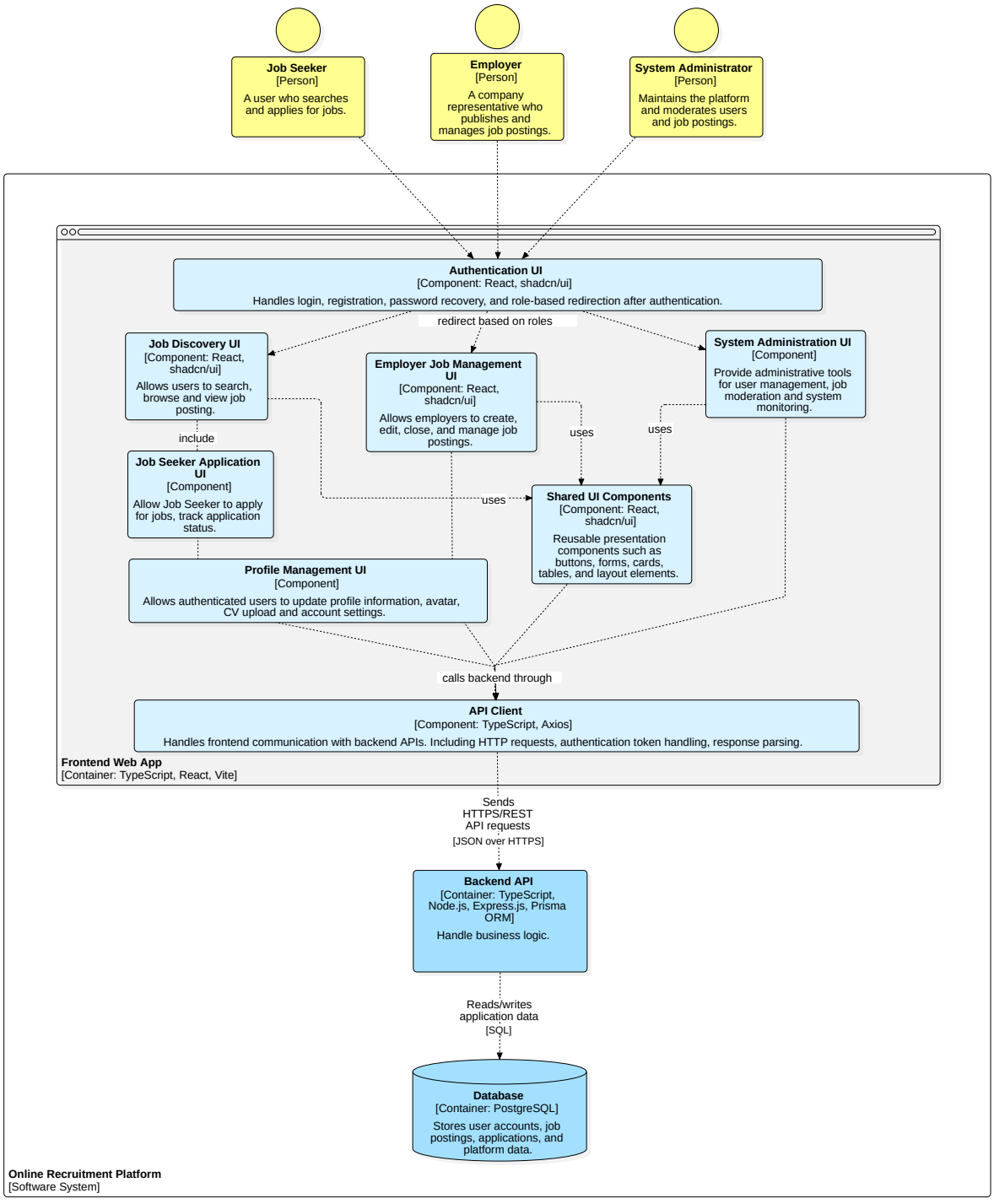

Figure 4.3: Frontend Component Diagram.

As shown in Figure 4.3, the frontend is organized around role-oriented feature components:

- The Authentication UI handles login, registration, and role-based redirection.
- the **Job Discovery UI** and **Job Seeker Application UI** support job browsing and application submission.
- the **Profile Management UI** manages personal information, avatar, and resumes.
- The Employer Job Management UI supports job posting management.
- The System Administration UI provides moderation tools.

All feature components reuse a common Shared UI Components library and communicate with the backend exclusively through a centralized API Client, which handles HTTP requests, authentication tokens, and response parsing.

4.2.4 Component View: Backend

The Backend component diagram decomposes the Backend API container into functional components that implement the platform's business capabilities.

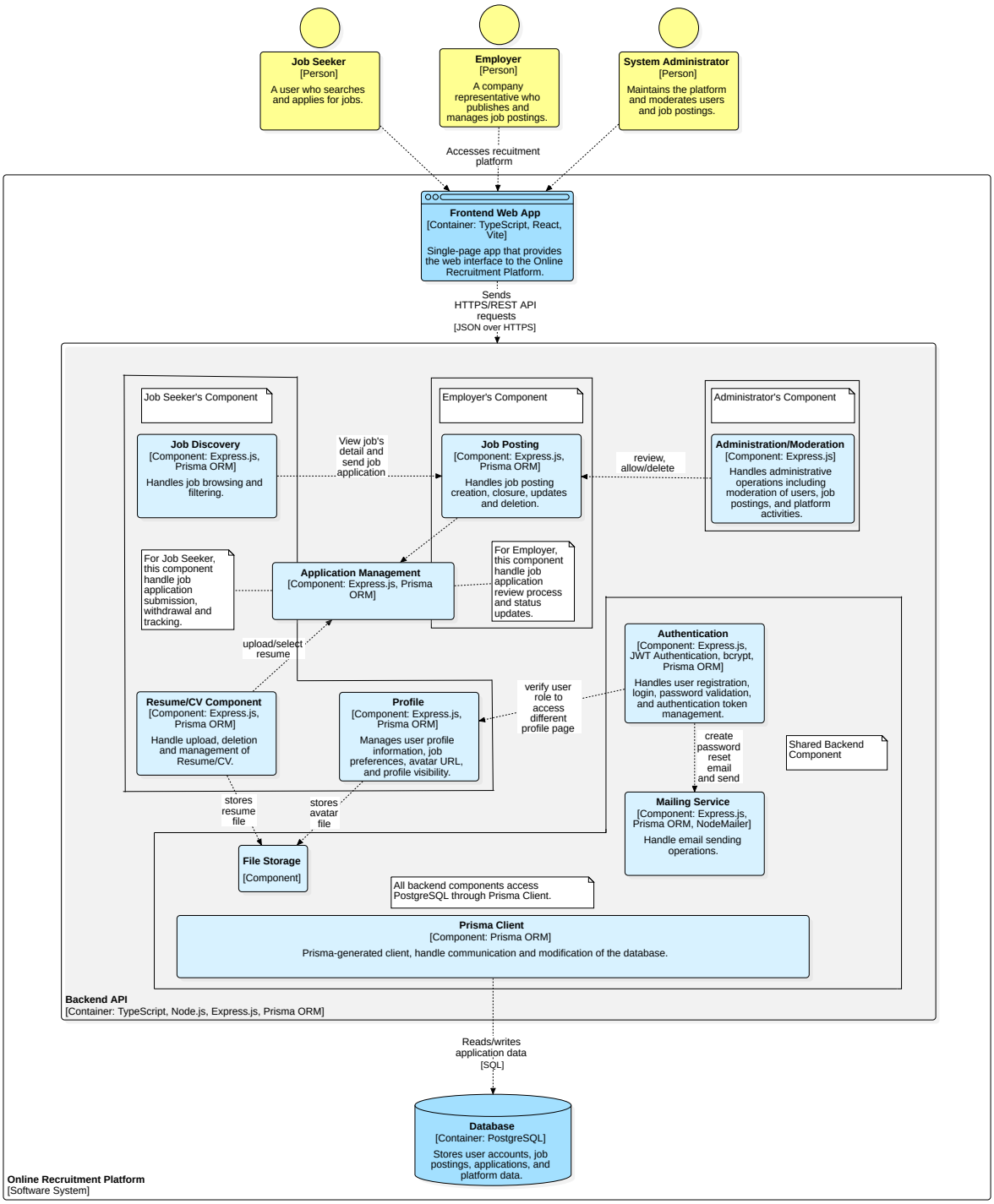

Figure 4.4: Backend Component Diagram.

As illustrated in Figure 4.4, the backend is organized into role-oriented business components.

- The Job Discovery, Profile, Resume/CV, and Application Management components support the Job Seeker's workflow.
- The Job Posting and Application Management components provide employers with job publishing and application review capabilities.

- The Administration/Moderation component enables platform administrators to manage users and job postings.

Common functionality is centralized in shared infrastructure components. The Authentication component manages user registration, login, and authorization using JWT-based authentication, while the Mailing Service handles email notifications. File assets are stored through the File Storage component.

All business components interact with the PostgreSQL database through the shared **Prisma Client**, ensuring a consistent data access mechanism across the application.

4.3 Database Design

The database design defines how the system's persistent data is organized and managed. It identifies the main business entities, their relationships, and the constraints that ensure data consistency. The design follows the principles introduced in Section 4.1, particularly data integrity, by using a normalized relational schema with clearly defined primary and foreign key relationships.

4.3.1 Entity Relationship Diagram

Figure 4.5 presents an overview of the database schema used by the Online Recruitment Platform. To improve readability, the overview diagram includes the primary entities, key attributes, and relationships while omitting system-generated fields such as timestamps and audit metadata.

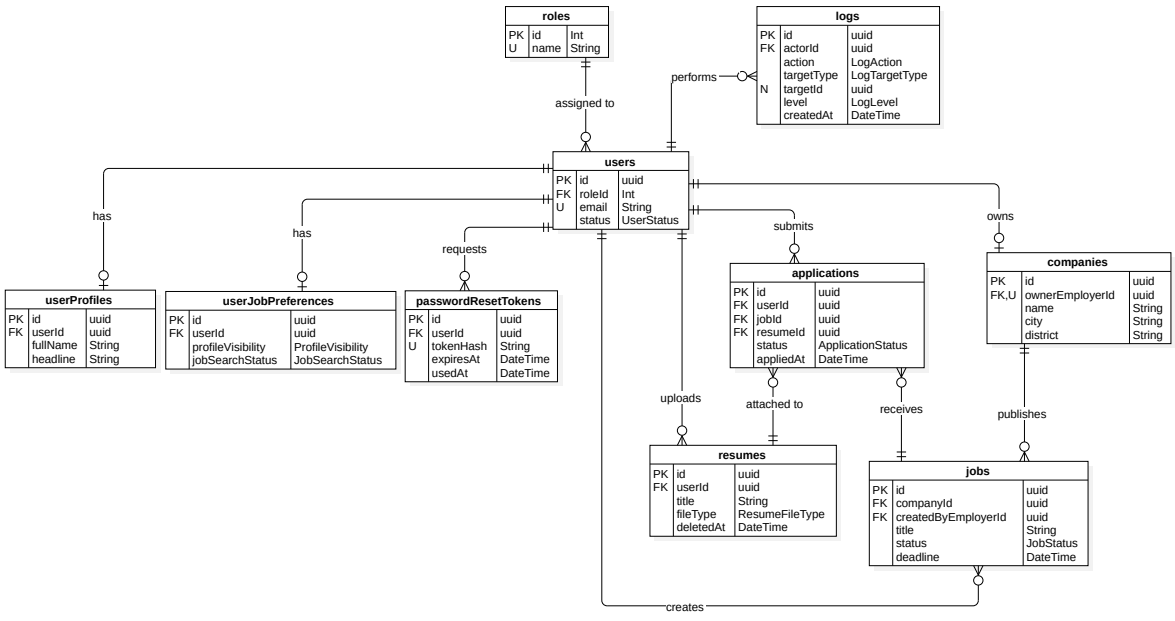

Figure 4.5: Entity Relationship Diagram.

The complete schema is organized into five functional groups, each corresponding to a major business domain and discussed in the following subsections.

Auth and Account

Figure 4.6 details the entities involved in authentication and account recovery.

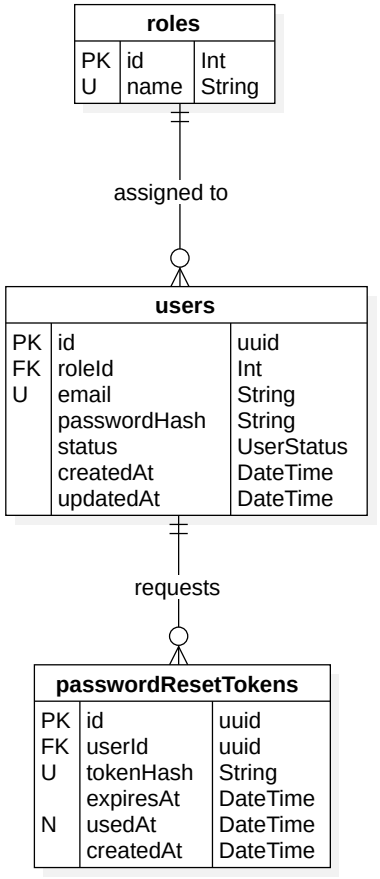

Figure 4.6: Auth and Account Entity Relationship Diagram.

Each role may be assigned to any number of users, while every user is assigned exactly one role. A user may request multiple password reset tokens over time, but each token belongs to exactly one user.

Profile and CV Management

Figure 4.7 details the entities involved in managing a user's profile, job preferences, and resumes.

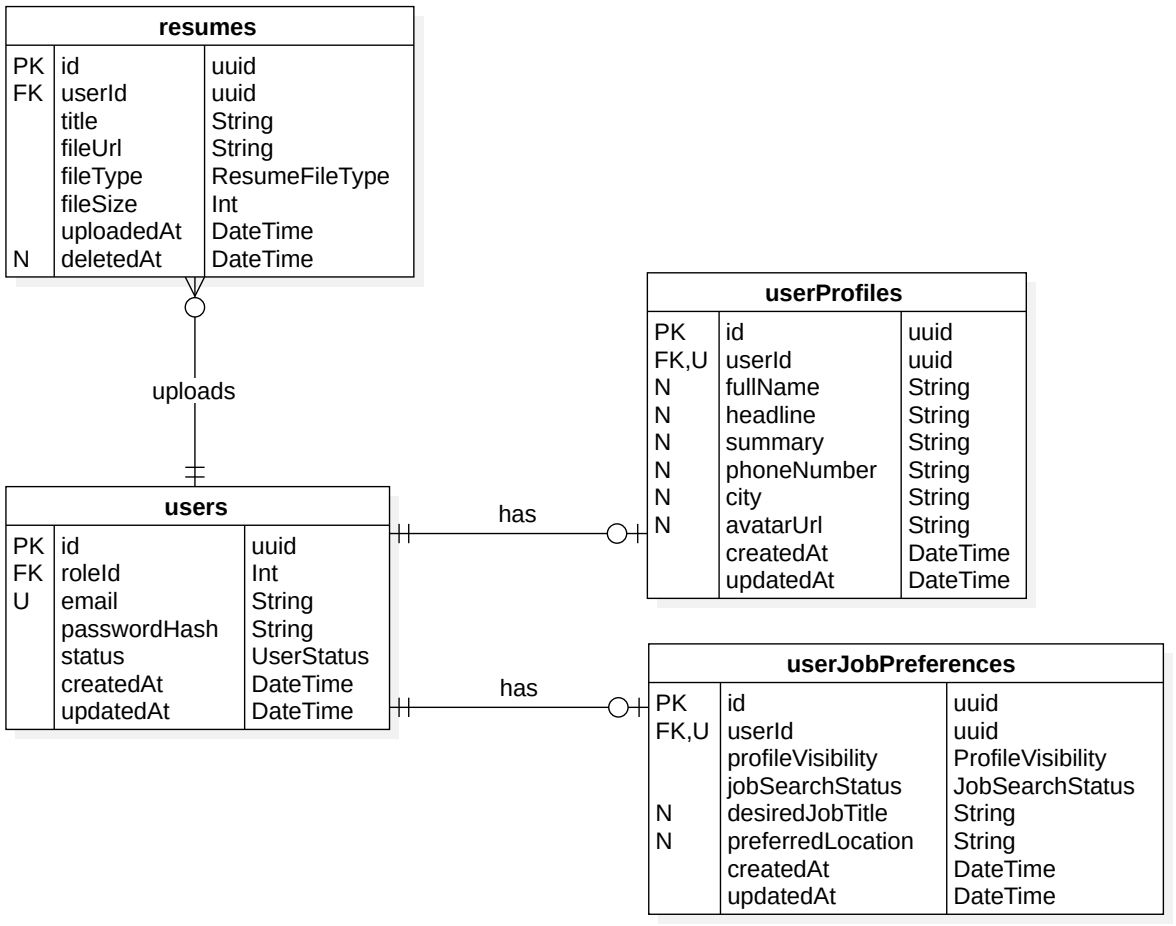

Figure 4.7: Profile and CV Management Entity Relationship Diagram.

Each user has at most one profile and at most one job preference record, both enforced through a unique foreign key, forming one-to-one relationships. A user may upload multiple resumes, but each resume belongs to exactly one user.

Company and Job Posting

Figure 4.8 details the entities involved in company and job posting management.

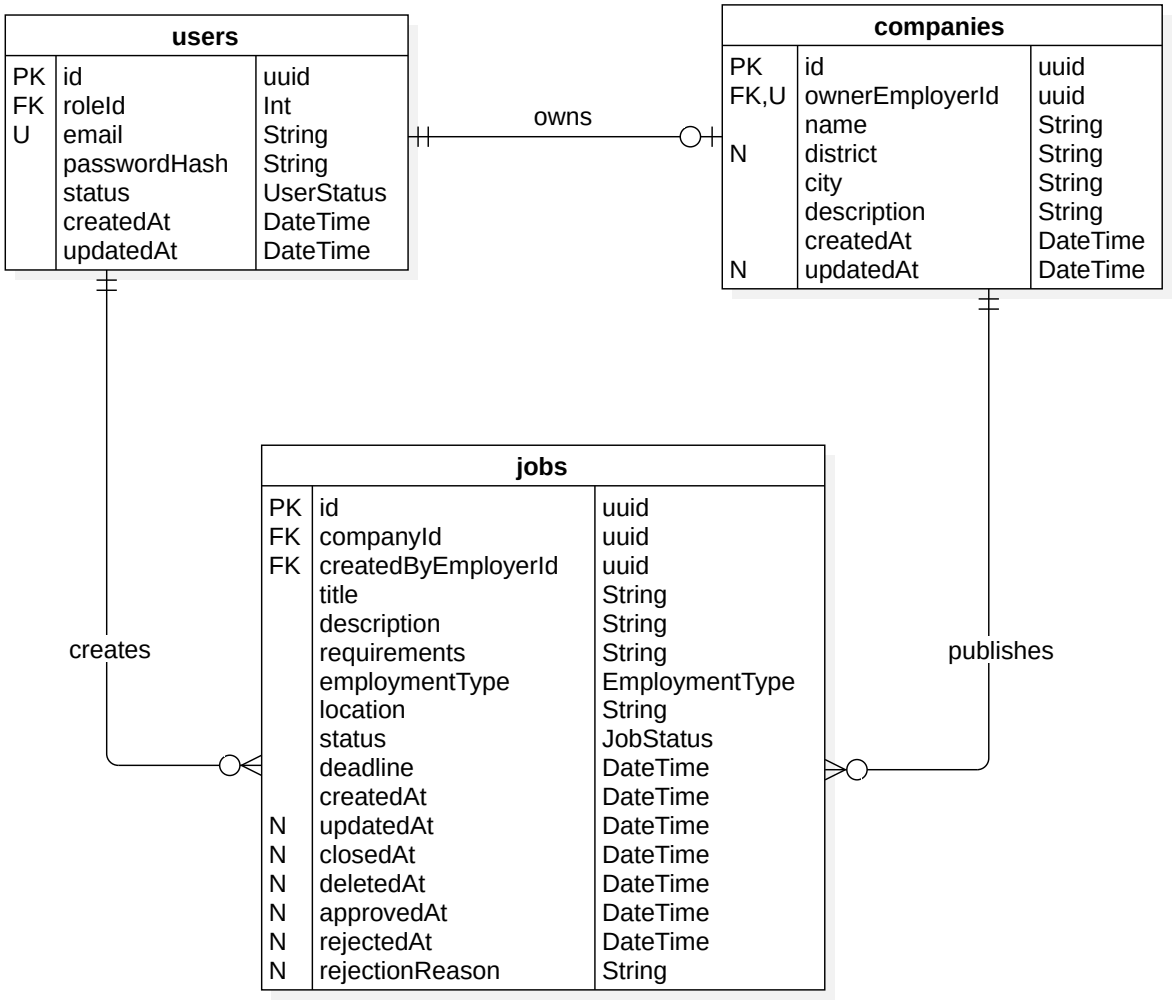

Figure 4.8: Company and Job Posting Entity Relationship Diagram.

An employer may own multiple companies, but each company belongs to exactly one employer. Similarly, an employer may create multiple job postings, but each job posting is created by exactly one employer. Each job posting belongs to exactly one company, while a company may publish any number of job postings.

Job Application

Figure 4.9 details the entities involved in the job application workflow.

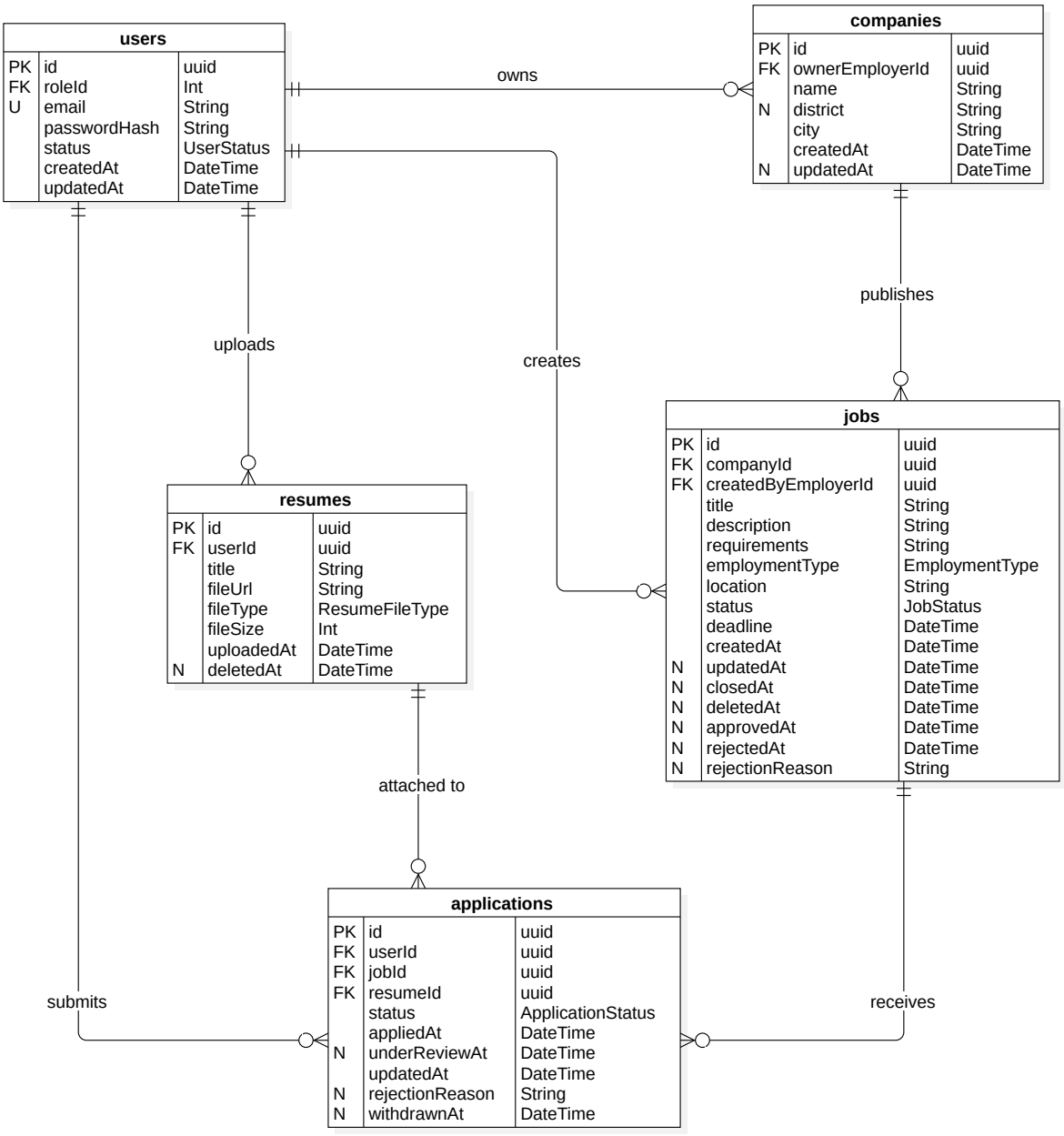

Figure 4.9: Job Application Entity Relationship Diagram.

A job seeker may upload multiple resumes and submit multiple applications, but each resume and each application belongs to exactly one job seeker. Each application is submitted for exactly one job posting, while a job posting may receive any number of applications. A resume may be attached to multiple applications, but each application references exactly one resume.

Administration and Audit

Figure 4.10 details the entities involved in role assignment and system activity logging.

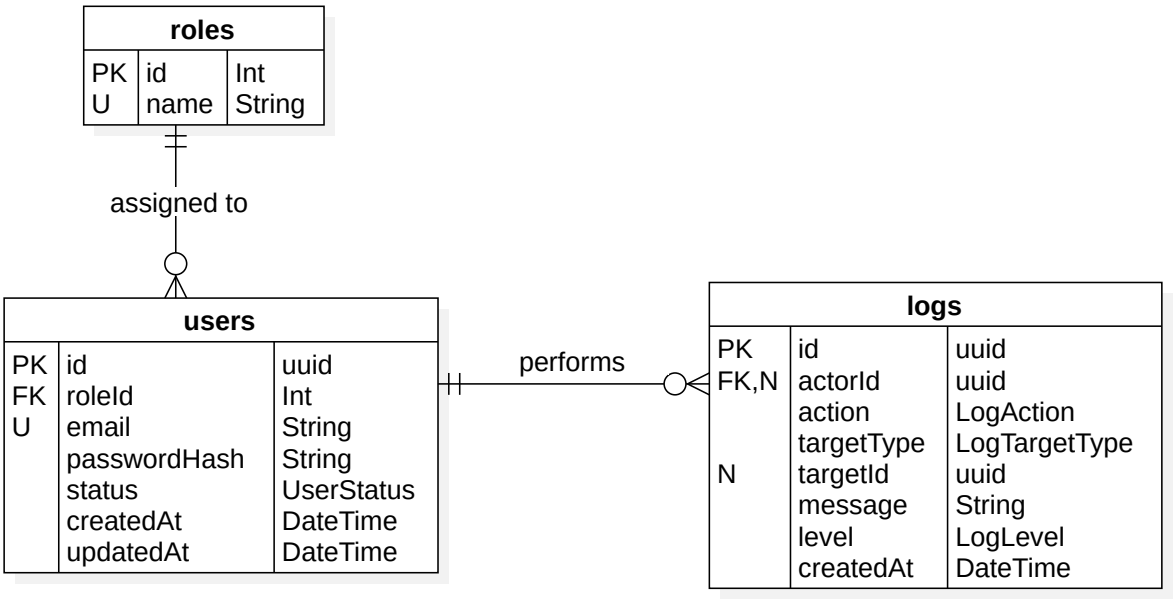

Figure 4.10: Administration and Audit Entity Relationship Diagram.

As with the Auth and Account group, each role may be assigned to any number of users, while every user is assigned exactly one role. A user may perform any number of logged actions, and each log entry optionally references the user who performed it, since some actions may be attributed to the system itself rather than a specific user.

4.4 Backend Design

This section details the internal design of the Backend API container introduced in Section 4.2. It first describes the project structure and shared infrastructure common to all modules, then presents each business module following the same template: an introduction, a class diagram, and a design explanation.

4.4.1 Backend Project Structure

The backend source code is organized as follows:

```
src
├── api-external
├── api-shared
├── api-internal
├── generated
├── lib
├── middlewares
└── utils
```

Table 4.2 summarizes the purpose of each top-level folder.

Table 4.2: Backend Project Structure.

Folder	Purpose
api-external	Implements business modules that expose application functionality to external users through RESTful APIs.
api-internal	Implements internal modules that support administrative and platform management operations.
api-shared	Provides reusable modules and services shared across multiple business domains to reduce duplication and improve maintainability.
generated	Contains source files automatically generated during the build process and should not be modified manually.
lib	Maintains shared library instances and application-wide configurations used by multiple modules.
middlewares	Implements request-processing components that perform cross-cutting concerns such as authentication and validation before controller execution.
utils	Provides general-purpose utility classes and helper functions used throughout the backend application.

4.4.2 Shared Backend Components

A small set of reusable infrastructure components is shared across all business modules, avoiding duplication and ensuring consistent behavior.

- JWT Authentication: issues and verifies signed JSON Web Tokens carrying the user's identifier and role, and is enforced through an Express middleware on protected routes.
- Email Service: wraps NodeMailer to send transactional emails, such as password reset notifications.
- PasswordHasher: wraps BCrypt to hash and compare passwords consistently wherever credentials are handled.
- ResetTokenUtils: generates password reset tokens, hashes them before storage, and validates expiration on use.
- Prisma Client: the generated ORM client, instantiated once and imported by services for all database access.
- File Storage: abstracts file persistence behind a common interface, used for avatar and resume uploads.

4.4.3 Authentication Module

Introduction

The Authentication Module implements user registration, login, logout, and password recovery. It resides in `api-external`, since these operations are invoked directly by external clients, and is one of the shared backend components other modules depend on for issuing and validating access tokens.

Class Diagram

Figure 4.11 presents the class diagram of the Authentication Module.

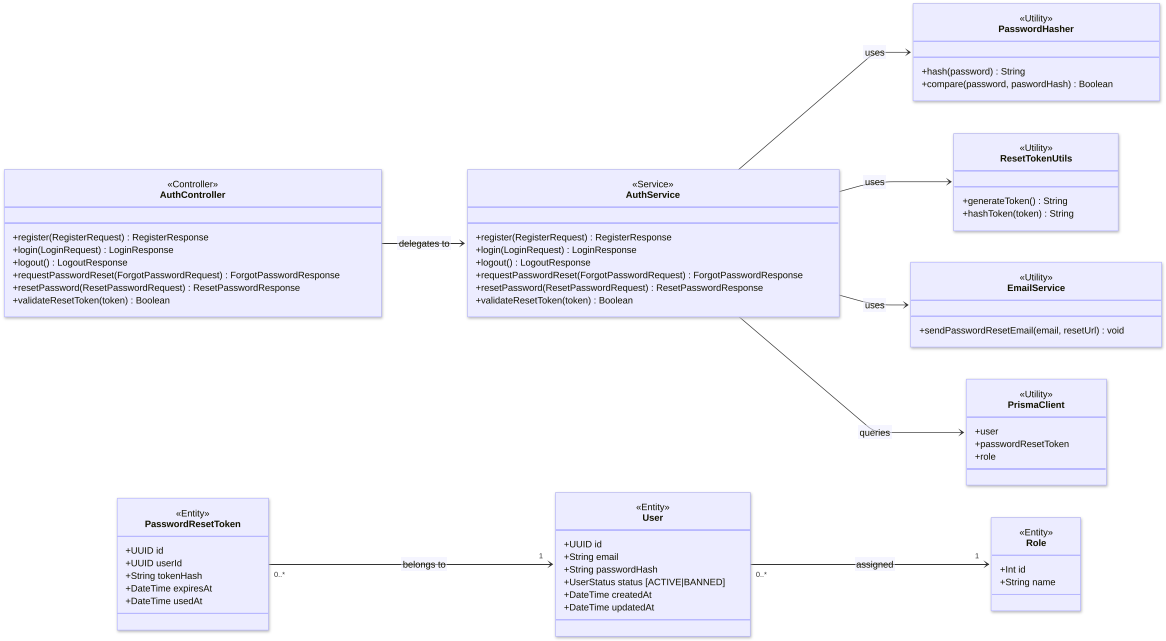

Figure 4.11: Authentication Module Class Diagram.

Design Explanation

Following the thin controller, rich service principle from Section 4.1, AuthController exposes six REST endpoints and delegates each directly to a matching method on AuthService, without containing business logic itself. AuthService depends on four shared utility components, described in Section 4.4.2: PasswordHasher, ResetTokenUtils, EmailService, and PrismaClient, the last of which queries and updates the User, PasswordResetToken, and Role entities whose relationships are detailed in Figure 4.6.

Each AuthService method implements one authentication use case:

- register validates that the email is not already in use, assigns the default JOB_SEEKER role by looking it up by name rather than a hardcoded identifier, and stores the hashed password.
- login verifies the submitted password against the stored hash, rejects banned accounts, and issues a signed JWT containing the user's identifier and role.
- logout returns a simple confirmation message, since JWTs are stateless and the actual token discarding is handled by the frontend.
- requestPasswordReset issues a hashed, time-limited reset token and emails the corresponding raw token to the user. To avoid revealing whether an email is registered, the same response message is returned regardless of whether the account exists.
- resetPassword validates the submitted token against the stored hash and expiration, then updates the user's password and marks the token as used within a single database transaction, ensuring both changes succeed or fail together.

- validateResetToken allows the frontend to check whether a reset token is still valid before rendering the reset password form, without exposing whether that check has consumed the token.

4.4.4 Profile Management Module

Introduction

The Profile Management Module lets authenticated users view and update their personal profile, job search preferences, avatar, and account password. It resides in `api-external` and depends on the Authentication Module's JWT middleware to identify the requesting user on every operation.

Class Diagram

Figure 4.12 presents the class diagram of the Profile Management Module.

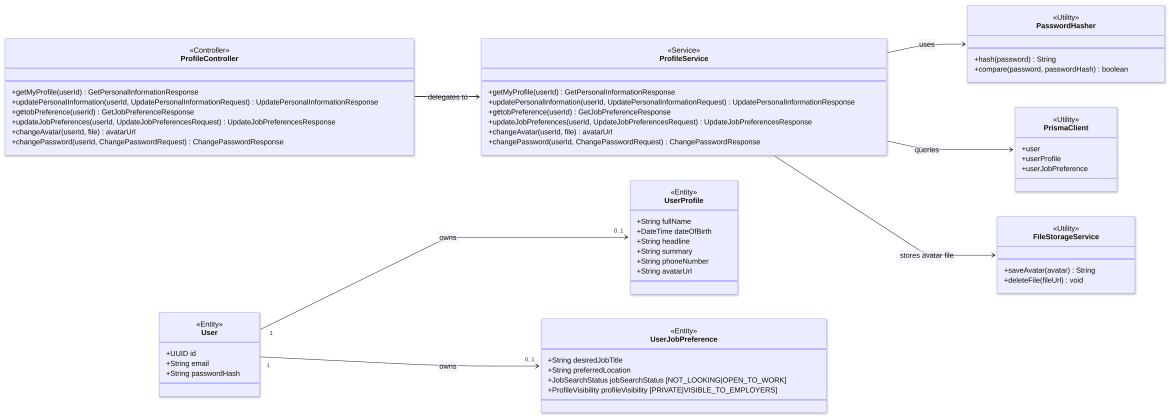

Figure 4.12: Profile Management Module Class Diagram.

Design Explanation

As with the Authentication Module, ProfileController delegates each of its six endpoints directly to a matching method on ProfileService, with no business logic in the controller itself. ProfileService depends on three shared utility components, described in Section 4.4.2: PasswordHasher, FileStorageService, and PrismaClient, the last of which reads and writes the User, UserProfile, and UserJobPreference entities whose relationships are detailed in Figure 4.7.

Each ProfileService method implements one profile management use case:

- getMyProfile and getJobPreference return the authenticated user's profile and job preference records respectively, selecting only fields safe for display; both may be empty if the user has not yet filled them in.
- updatePersonalInformation and updateJobPreference use an upsert operation, creating the record on first use and updating it thereafter, so the client does not need to know in advance whether a profile already exists.
- updateJobPreference additionally distinguishes between an omitted field, which is left unchanged, and a field explicitly set to null, which clears it, by checking

each field against undefined before including it in the update. This allows partial updates without unintentionally overwriting fields the client did not intend to modify.

- changeAvatar validates the uploaded file's type and size before storing it, rejecting formats other than JPG, JPEG, or PNG and files larger than 3 MB, then updates the user's avatar URL.
- changePassword verifies the submitted current password, rejects the request if the new password is identical to the old one, and otherwise hashes and stores the new password.

Across all methods, only the fields explicitly listed in each Prisma `select` clause are returned or made editable; protected fields such as role, account status, and password hash are never exposed through this module, consistent with the requirement that users may modify only their own non-protected profile data.

4.4.5 Resume Management Module

Introduction

The Resume Management Module lets job seekers list, upload, and delete their resumes, which are later selected when submitting job applications. It resides in `api-external` and depends on the Authentication Module's JWT middleware to identify the requesting user on every operation.

Class Diagram

Figure 4.13 presents the class diagram of the Resume Management Module.

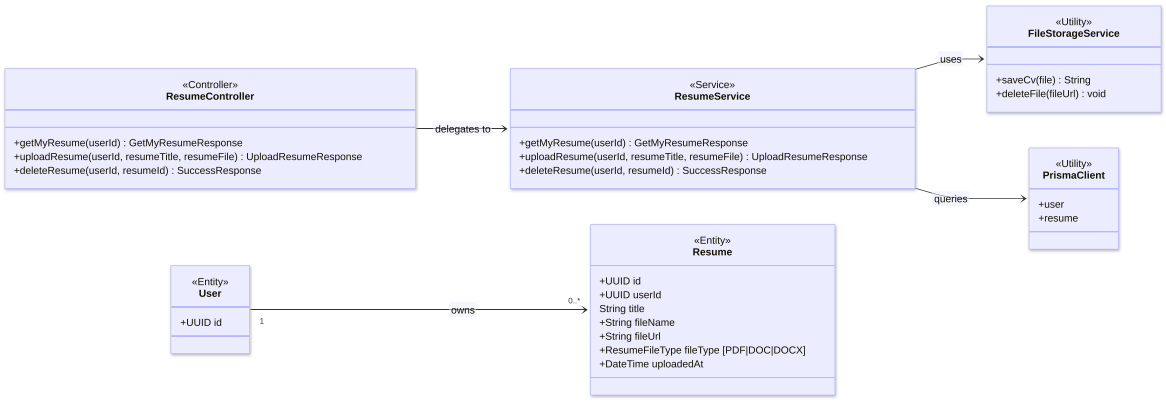

Figure 4.13: Resume Management Module Class Diagram.

Design Explanation

As with the previous modules, ResumeController delegates each of its three endpoints directly to a matching method on ResumeService, with no business logic in the controller itself. ResumeService depends on two shared utility components, described in Section 4.4.2: FileStorageService and PrismaClient, the latter of which reads and writes the User and Resume entities, whose relationship is detailed in Figure 4.7.

Each ResumeService method implements one resume management use case:

- getMyResumes returns the authenticated user's non-deleted resumes, ordered from newest to oldest.
- uploadResume validates that a title and file are provided, rejects files larger than 5 MB or of an unsupported type, accepting only PDF, DOC, and DOCX, then stores the file through FileStorageService before creating the resume record.
- deleteResume verifies that the resume exists, is not already deleted, and belongs to the requesting user, then checks whether it is referenced by any application. If it is, the resume is soft-deleted, marked with a deletion timestamp but kept in the database, so that existing applications retain a valid reference. Otherwise, it is hard-deleted from both the database and file storage.

4.4.6 Job Discovery Module

Introduction

The Job Discovery Module lets guests and authenticated users search active job postings and view their details. Unlike the previous modules, its endpoints are not protected by the JWT middleware, since job browsing is a public capability. It resides in `api-external`.

Class Diagram

Figure 4.14 presents the class diagram covering the Public actor's capabilities, job discovery and company profile viewing. This section discusses only JobDiscoveryController and JobDiscoveryService; CompanyController and CompanyService are discussed in Section 4.4.8 alongside the rest of the system's company-related design.

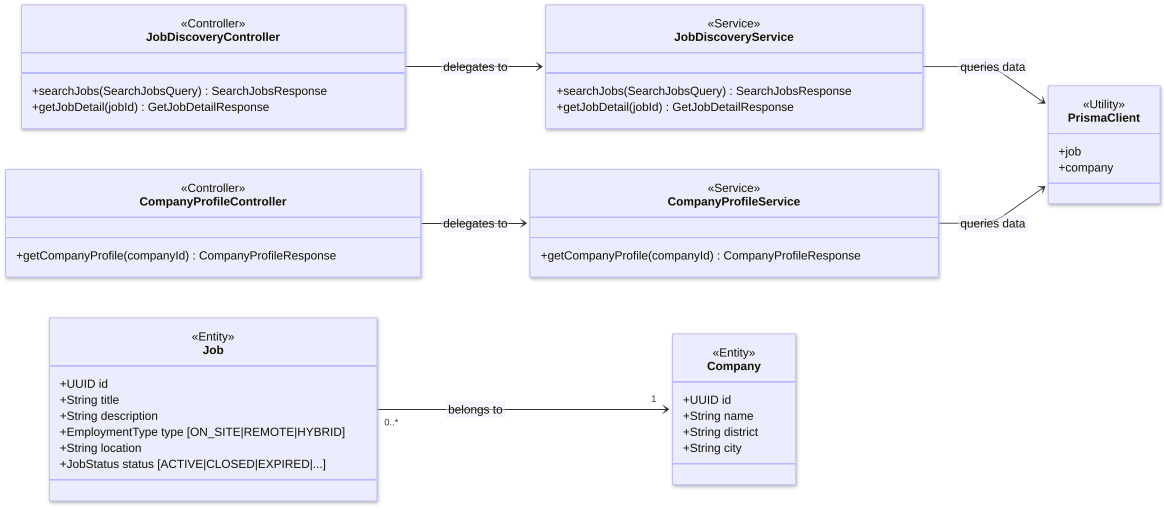

Figure 4.14: Public Class Diagram.

Design Explanation

As with the previous modules, JobDiscoveryController delegates each of its two endpoints directly to a matching method on JobDiscoveryService, with no business logic

in the controller itself. JobDiscoveryService depends on the shared PrismaClient, described in Section 4.4.2, which queries the Job and Company entities, whose relationship is detailed in Figure 4.8.

Each JobDiscoveryService method implements one job discovery use case:

- searchJobs restricts results to ACTIVE jobs and accepts an optional keyword and location, either of which may be provided independently or together. An empty string is treated as not provided, and each value is capped at 255 characters. When given, the keyword matches against the job title or company name, and the location matches against the job's own location field or the company's city and district, all case-insensitively. When both are provided, a job must satisfy both conditions to be returned. If neither is provided, all ACTIVE jobs are returned, and a query that matches nothing returns an empty list rather than an error.

- getJobDetail returns the full detail of a single job by identifier, restricted to ACTIVE jobs, and returns a 404 error if the job does not exist or is not currently active.

4.4.7 Job Application Module

Introduction

The Job Application Module lets job seekers apply to active job postings using one of their resumes, view their own submitted applications, and withdraw an application while it is still eligible. It resides in `api-external`, and every endpoint requires both a valid JWT and the JOB_SEEKER role.

Class Diagram

Figure 4.15 presents the class diagram of the Job Application Module.

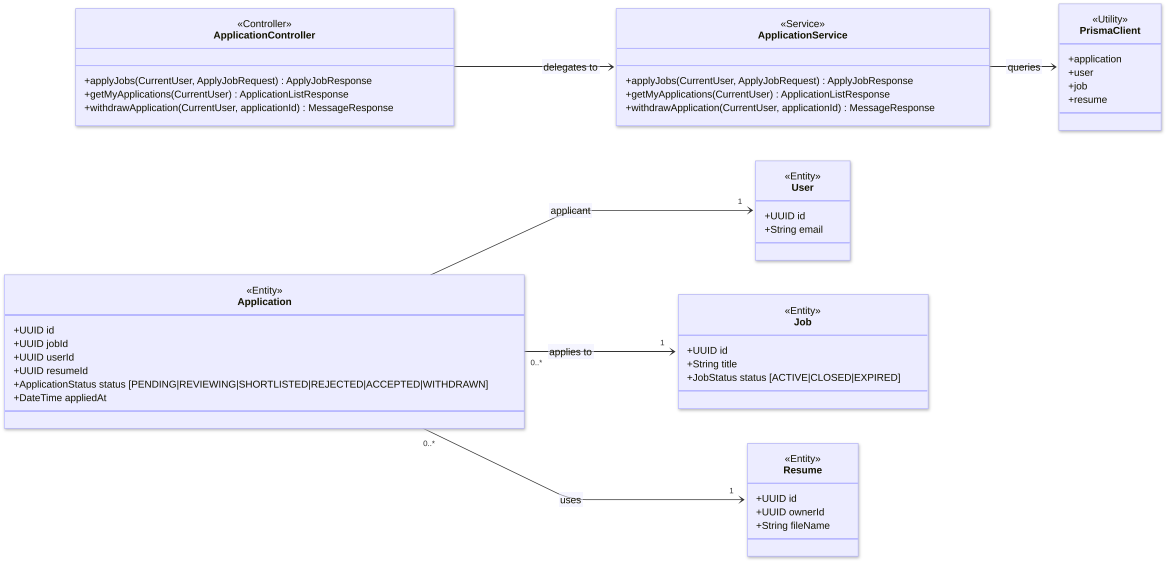

Figure 4.15: Job Application Module Class Diagram.

Design Explanation

As with the previous modules, ApplicationController delegates each of its three endpoints directly to a matching method on ApplicationService, with no business logic in the controller itself. ApplicationService depends on the shared PrismaClient, described in Section 4.4.2, which queries the User, Job, Resume, and Application entities, whose relationships are detailed in Figure 4.9.

Every ApplicationService method begins with the same guard: the caller must hold the JOB_SEEKER role and must not be banned, otherwise the request is rejected. Beyond this shared guard, each method implements one job application use case:

- applyJob verifies that the referenced job exists and is ACTIVE, that the referenced resume exists and belongs to the caller, and that no prior application exists for the same job and user pair, before creating the application with an initial status of SUBMITTED.
- getMyApplications returns all applications submitted by the caller, joining through the related job and company to return a flattened summary alongside each application's status and submission date.
- withdrawApplication verifies that the application belongs to the caller and is still in SUBMITTED status, rejecting the request once it has moved further along the review process, then updates its status to WITHDRAWN and records the withdrawal timestamp.

4.4.8 Employer Job Management Module

Introduction

The Employer Job Management Module covers the three capabilities exposed to Employers: managing their company profile, managing the lifecycle of their job postings, and reviewing submitted applications. It also includes the public-facing company profile and active job listing endpoints introduced in Section 4.4.6, since they operate on the same Company data. Implementation of this module is still in progress at the time of writing; the design below reflects the structure defined by the class diagrams and will be finalized once implementation is complete.

Class Diagram

Figures 4.16, 4.17, and 4.18 present the class diagrams of the Employer Job Management Module.

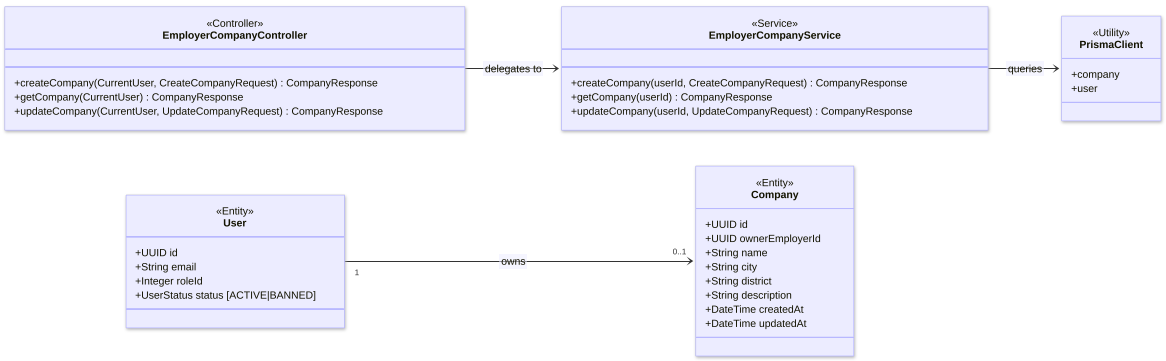

Figure 4.16: Company Profile Management Class Diagram.

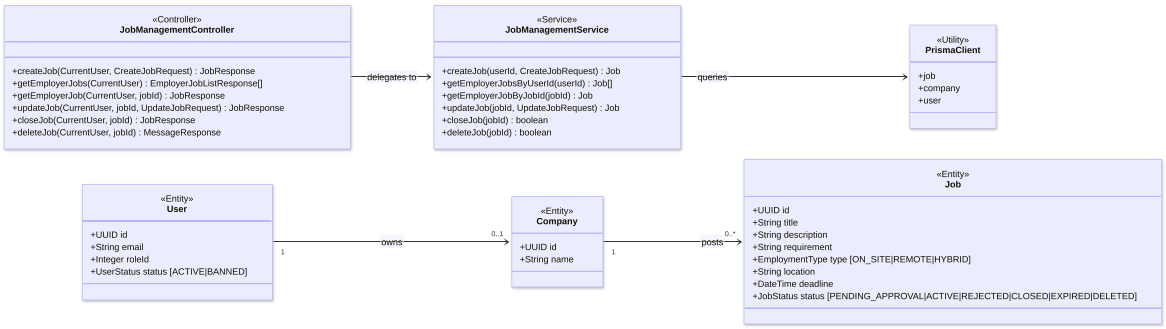

Figure 4.17: Job Posting Management Class Diagram.

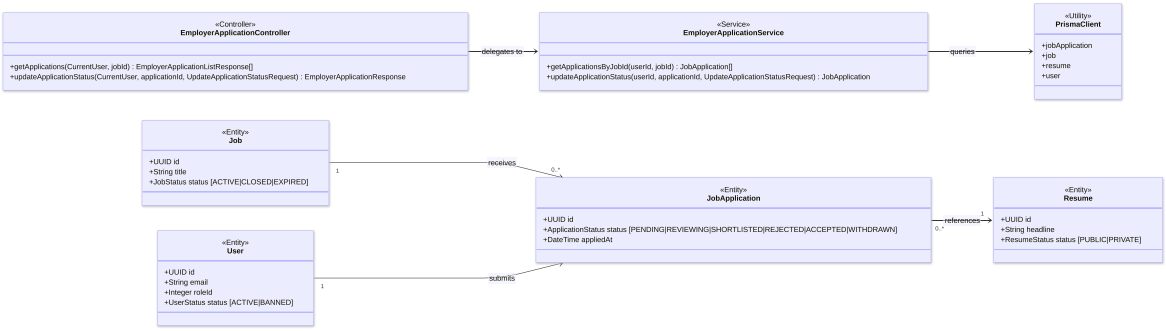

Figure 4.18: Employer Application Management Class Diagram.

Design Explanation

Across all three diagrams, each Controller delegates directly to a matching Service method, with no business logic in the controller itself, and each Service depends on the shared PrismaClient described in Section 4.4.2.

Company Profile Management EmployerCompanyController exposes createCompany, getCompany, and updateCompany, delegated to EmployerCompanyService. As shown in Figure 4.16, an employer owns at most one company, so none of these operations take a company identifier; the owning employer is instead resolved from the authenticated user.

Company Profile Viewing The public-facing counterpart, CompanyController and CompanyService, was introduced alongside the Job Discovery Module in Figure 4.14. getCompanyProfile returns a company's public details, and getActiveJobs lists its ACTIVE job postings, letting guests and job seekers view a company without requiring authentication.

Job Posting Management JobManagementController exposes the full lifecycle of a job posting: createJob, getEmployerJobs, getEmployerJob, updateJob, closeJob, and deleteJob, delegated to JobManagementService. As shown in Figure 4.17, the Job status attribute includes PENDING_APPROVAL and REJECTED, in addition to the statuses used elsewhere in the system, reflecting that newly created and edited postings are subject to administrator moderation, detailed in Section **??**, before becoming publicly visible.

Application Review EmployerApplicationController exposes getApplications, which lists the applications submitted to a given job, and updateApplicationStatus, which transitions an application through the reviewing pipeline, delegated to EmployerApplicationService. As shown in Figure 4.18, this complements the SUBMITTED and WITHDRAWN transitions initiated by the job seeker in Section 4.4.7: here, the employer moves an application through the remaining REVIEWING, SHORTLISTED, REJECTED, and ACCEPTED statuses.

4.5 Frontend Design

4.6 Summary

Chapter 5

Implementation

Chapter 6

Testing

Chapter 7

Conclusion

Bibliography